



清华大学
Tsinghua University

RELICS延迟电子模拟

蔡畅

清华大学

2024. 5. 15

1



摘要

- 延迟电子来源；
- 模拟程序RELICS_DE_Sim介绍（输入输出，模拟逻辑，性能表现）；
- 模拟结果，以及对应筛选条件；
- 与波形筛选的联合；

延迟电子本底产生机制

- 较大的信号过后，TPC内部会存在一定量的电子经过时间延迟之后逐渐逸出汽液分解面，由于不同的延迟产生原因，这些电子往往具有不同的“寿命”，在时间序列上呈现为“随机分布”；
- 由于TPC中较小S2信号的波形的“离散化”特征，需要我们把一定时间间隔内的触发形成的波形信号记录并合并分析；

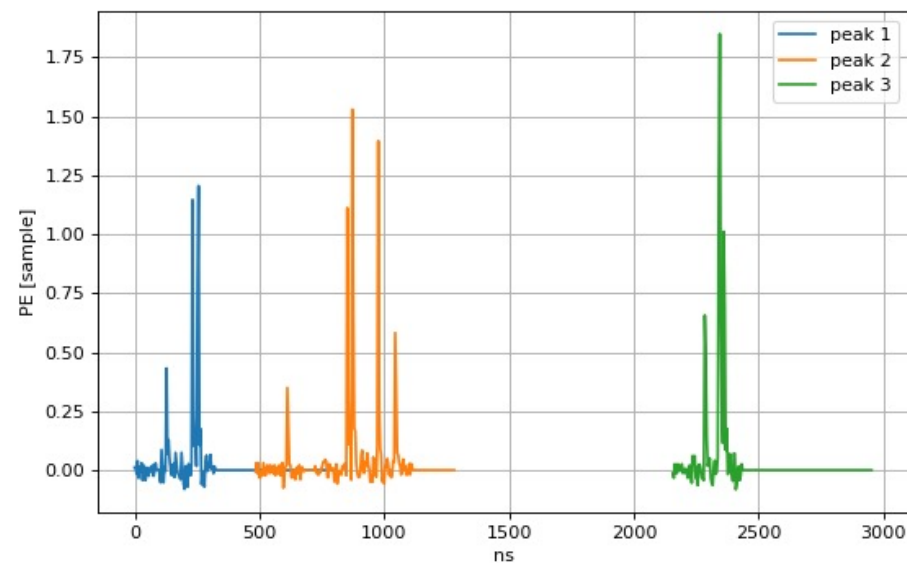
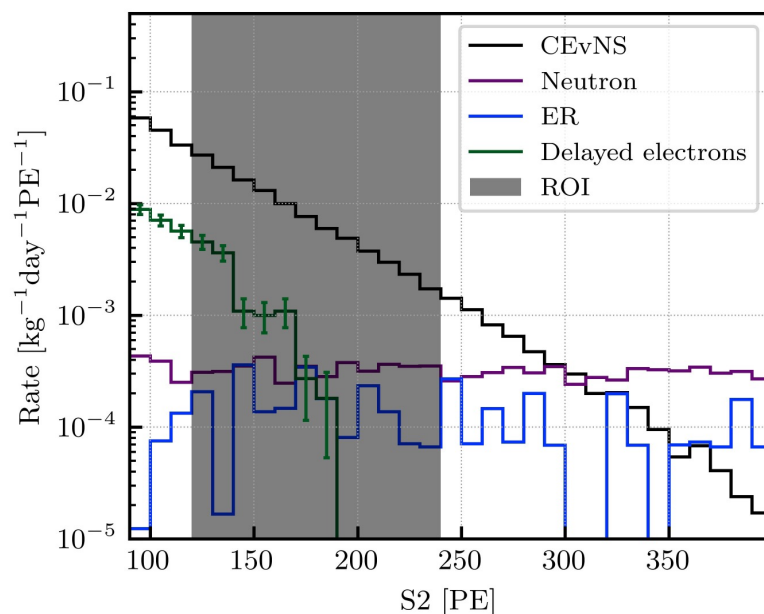


图2 RELICS原型机中临近时间多次触发波形图



模拟程序介绍

1. Muon沉积能量点
2. DE事件信息
3. Pile_up事件(新增双电子事件模拟)
4. S2 Pattern 模拟
5. 位置重建
6. 重建位置S2 Pattern再次模拟
7. CEvNS与Pile-up比较



GENERATE_MUON_POINTS ()

- 输入: `muon_file_list` (路径)
- 输出: `self.dense_muon_points`(numpy array)
- 详细内容:
 - 读取muon文件, 并生成相关信息:
 - 从每个muon_file中读取所有muon能量沉积点信息;
 - 计算muon的总数`merged_muon_array[ 'eventId' ][-1]+1)`, 折算模拟时间`time_range`, 并在`time_range`中均匀撒随机数为每个muon事件的时间点;
 - 计算每个能量沉积点对应的电子产额: `quenching_factor * merged_muon_array[ 'energy' ]`;
 - 根据muon信息, 生成插值能量沉积点信息(`dense_muon_points`) (20ms deadtime);
 - 从muon track的首尾线性插值, 并且能量均匀分布, 电子差额均匀分布;
 - 根据插值能量沉积点的深度, 对于每个插值能量沉积点, 计算对应的延迟电子数量(从2ms死时间到20ms死时间比例进行折算, $\gamma = -1.1$, $k = 0.587753$);

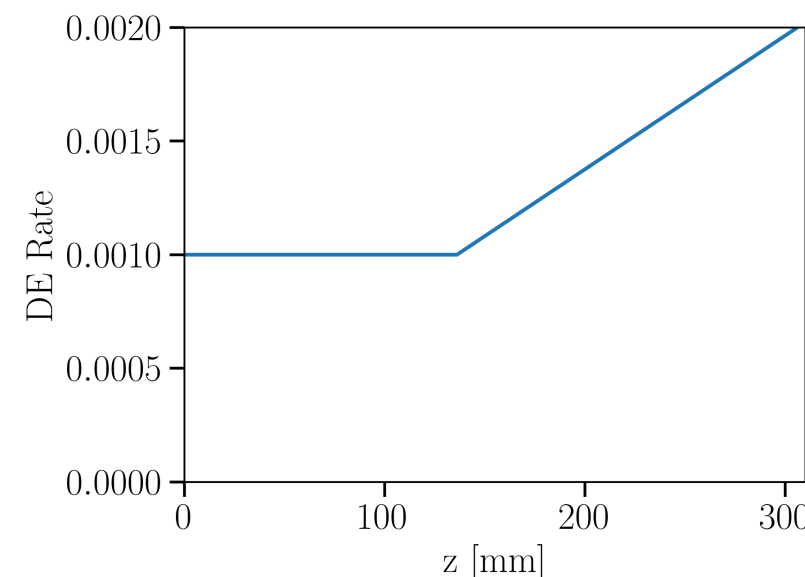


Fig1.DE Rate



GENERATE_MUON_POINTS ()

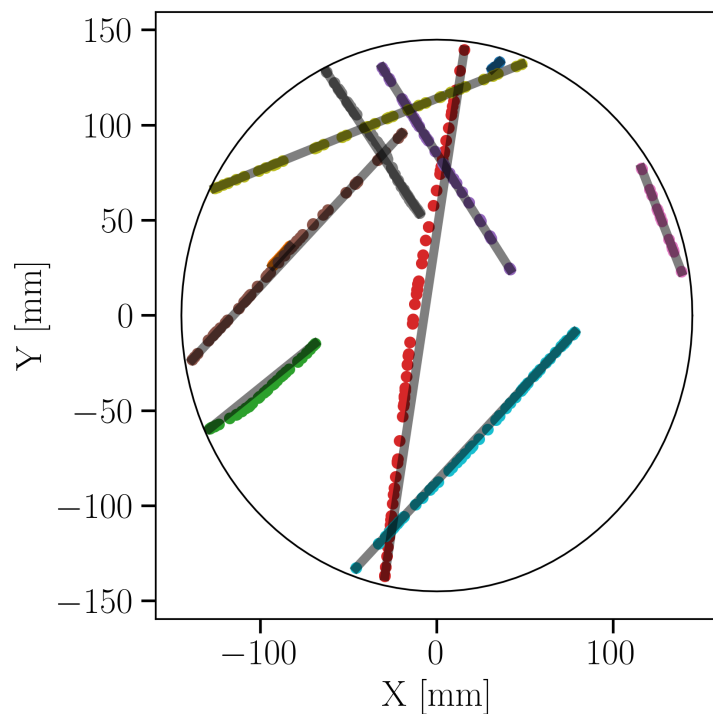


Fig2 Muon Track
灰色为插值后的track

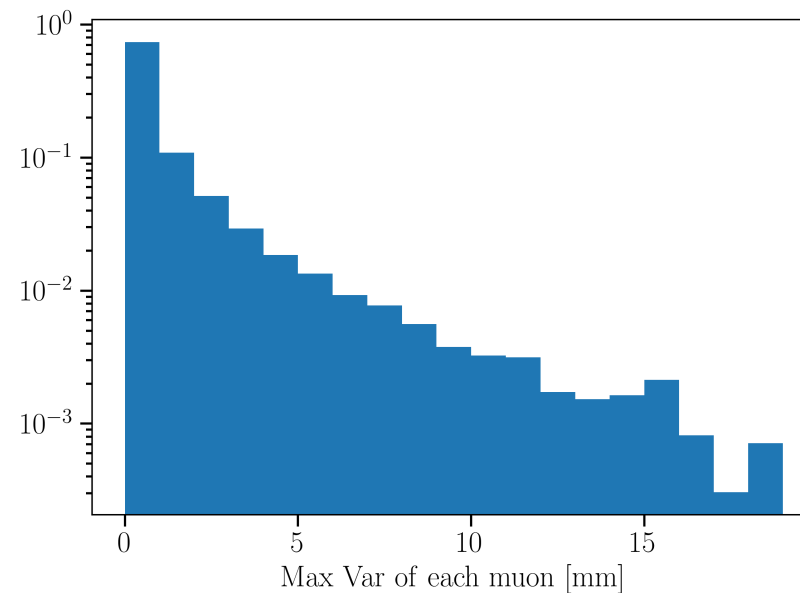


Fig3. Muon Track Var
每个muon track中与插
值结果最大距离

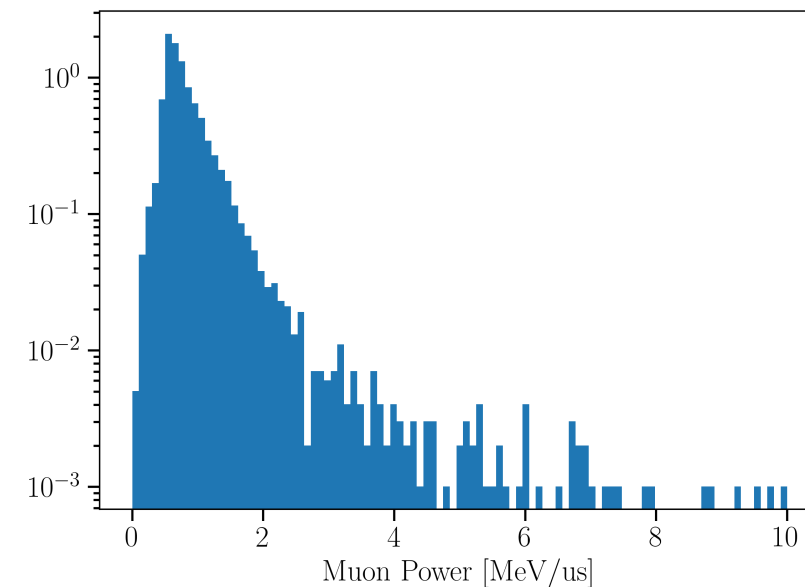


Fig4. Muon S2 Power
Muon平均S2信号对
应“功率”



GENERATE_DE_POINTS ()

- 输入: dense_muon_points, dead_time, diffuse_length
- 输出: delayed_electron(numpy array)
- 内容: 根据muon插值能量沉积点, 创建所有延迟电子的信息;
 - 根据muon插值能量沉积点记录的延迟电子数目, 生成对应数量的延迟电子的信息 array;
 - xd, yd 根据muon插值能量沉积的水平位置, 叠加上给定的diffuse_length的二维高斯偏差;
 - e_time 根据muon_time 叠加给定dead_time-2s内的power law随机分布, $\gamma = -1.1$;
 - 去除139mm以外的DE (这些电子过于靠近容器壁, 可能被吸附或者别的效应, 且根据光模拟, 容易有nan出现);
 - 对于所有延迟电子, 按照最终的e_time进行排序

```
[('eventId', '<i4'),  
 ('xd', '<f8'),  
 ('yd', '<f8'),  
 ('muon_time', '<f8'),  
 ('e_time', '<f8')]
```



GENERATE_PILE_UP_3E ()

- 以三电子事件生成为例
- 输入: `delayed_electron`
- 输出: `pile_up_e_num(list)` , `pile_up_result(list)` , `pile_up_indices(list)`
- 内容:
 - `delayed_electron[ 'e_time' ][1:] - delayed_electron['e_time'][:-1] < gap`
 - 连续3电子事件, 需要第一个与前一个不相连, 且后面两个电子均相连, 第三个电子与之后不相连;
 - 对应3电子事件, 组成 $N \times 3$ 的2d array, 加入 `pile_up_result`;

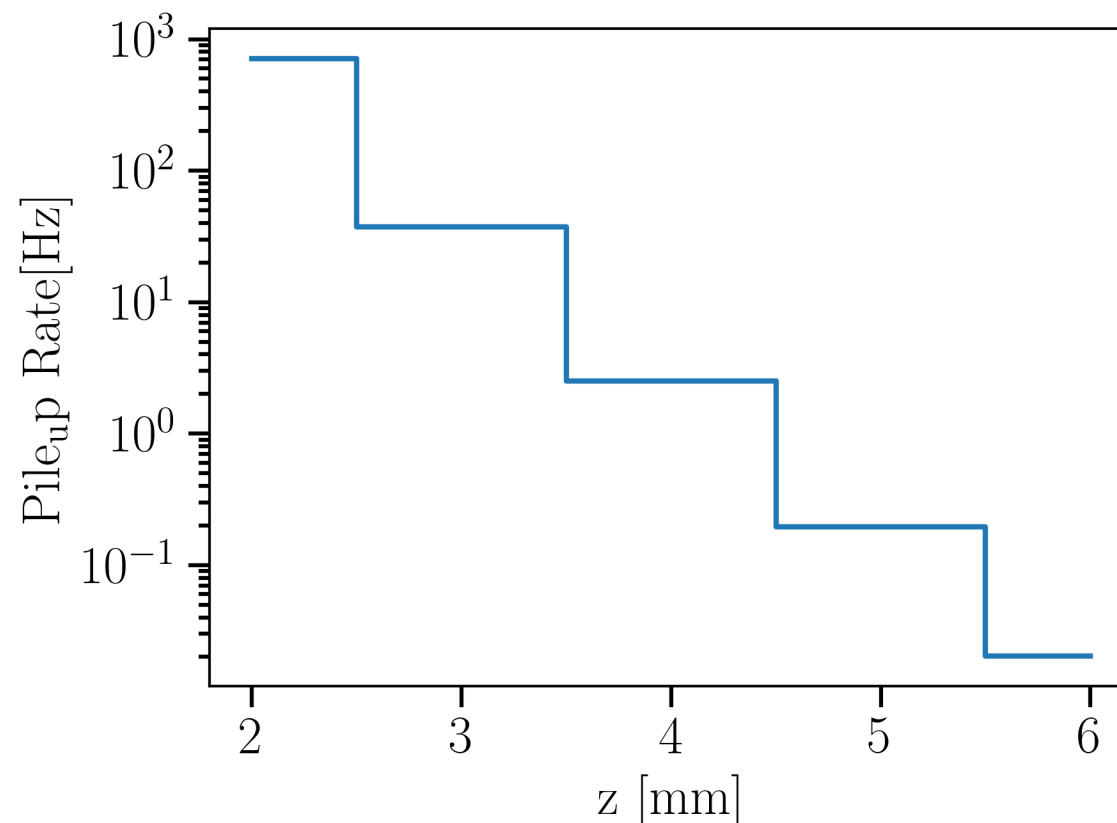


Fig5. Pile-up之后不同电子数事件的平均事例率, 加入双电子事件模拟后数据量再提升一个量级



GENERATE_DE_PATTERN()

- 输入: `pile_up_result`
- 输出:
 1. `pile_up_light_pattern` 128通道pe数期望值 (预期pe打到pmt的光阴级数量)
 2. `pile_up_pe_pattern` 128通道pe数离散值
 3. `pile_up_area_pattern` 128通道面积值
 4. `pile_up_pe_by_area` (`pile_up_area_pattern` / `pe_gain`) 128通道pe数期望值 (pe)
 5. `pile_up_pe_info` 所有被记录的pe的信息 (array)
- 内容: 模拟事件根据光模拟结果在通道的响应

```
[('event_id', 'uint64'),  
 ('area', '<f8'),  
 ('channel', 'uint16'),  
 ('e_time', '<f8'),  
 ('e_id', 'uint32')]
```



- 1. 在指定 $x = 1.0$, $y=1.5$ 单次撒 $100 * \text{np.random.normal}(1, 0.265, \text{len}(\text{pattern}))$ (需要调整) 1000次, 得到 $1000*128$ 的 `array1` ($1000*$ 每个通道接收到pe的数量)
- 2. `array1 * 每个通道的QE` (作为均值进行泊松分布的离散化) 得到 $1000*128$ 的 `array2` ($1000*128$ 的每个通道实际响应的pe数量)
- 3. `array2 * SinglePhotonelectron gain` 的误差响应, 得到 `array3`, 才是最后的结果
- 4. `array3` 应该是一个每行求和, 均值为30, std为10的状态



GENERATE_DE_PATTERN()

- 根据`pile_up_result`中包含的每个电子的坐标：
 - 每个通道会读取一个参数：`LCE_Value` (2d array), 以及对应事件位置`LCE_xs`, `LCE_ys`, 代表一个指定位置发出的光对应于该通道的采集效率；
 - 对于每个通道, 通过`LCE_Value`, `LCE_xs`, `LCE_ys`以及各个电子的水平位置, 可以模拟出每个事件位置对应的该通道pe数期望值 (每产生1pe), 对128个通道进行计算, 得到`pattern` ;
 - 根据导入的`single_electron_gain` (30pe) 以及设定的`sigma` (10pe), 通过`pattern`得到`alter_pattern`, 并调整小于0的通道, 最后泊松分布离散化得到`alter_pe_pattern` (对应探测器每个单电子信号生成过程中track随机性以及其它随机物理过程导致的光产额的整体变化) ;

```
# 放大SE响应的随机效应
pattern = DE_pile_up_pattern_original[i]
gamma = np.random.normal(1, 0.265, len(pattern))
alter_pattern = pattern * \
    self.single_electron_gain * gamma[:, np.newaxis]
alter_pattern[alter_pattern < 0] = 0 # 随机效应导致小于0期望值的修正
alter_pe_pattern = np.random.poisson(
    alter_pattern)
```



GENERATE_DE_PATTERN()

- 在得到`alter_pe_pattern`之后，我们可以开始针对pe的模拟(对每个event for循环)；

- `channel_indices = np.repeat(np.arange(128), fake_pe_pattern[j])`

- 再根据给定的`pe_gain`以及`pe_gain_std`可以模拟每个pe对于pmt响应随机化后的读出；

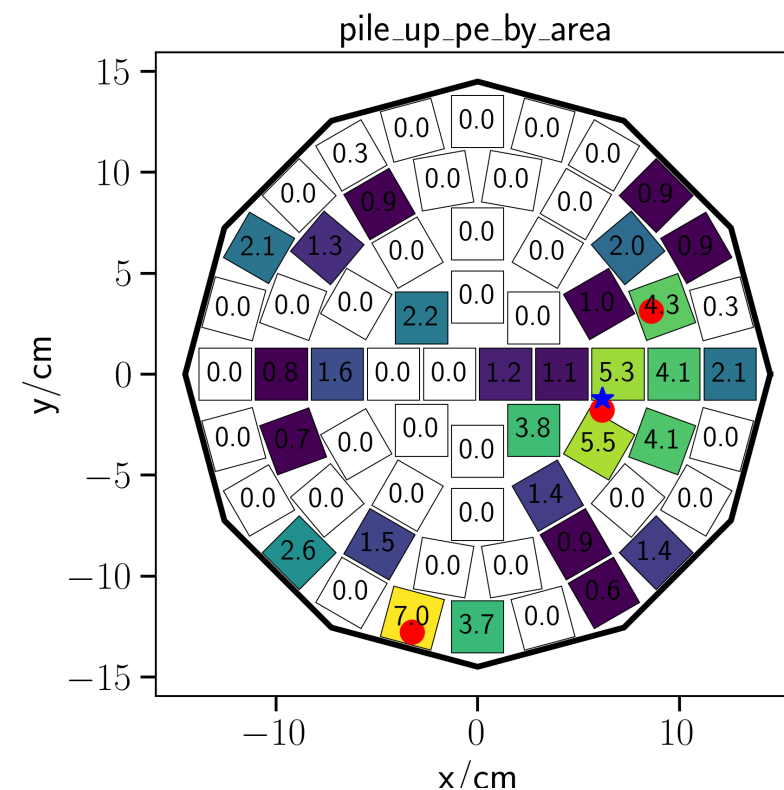
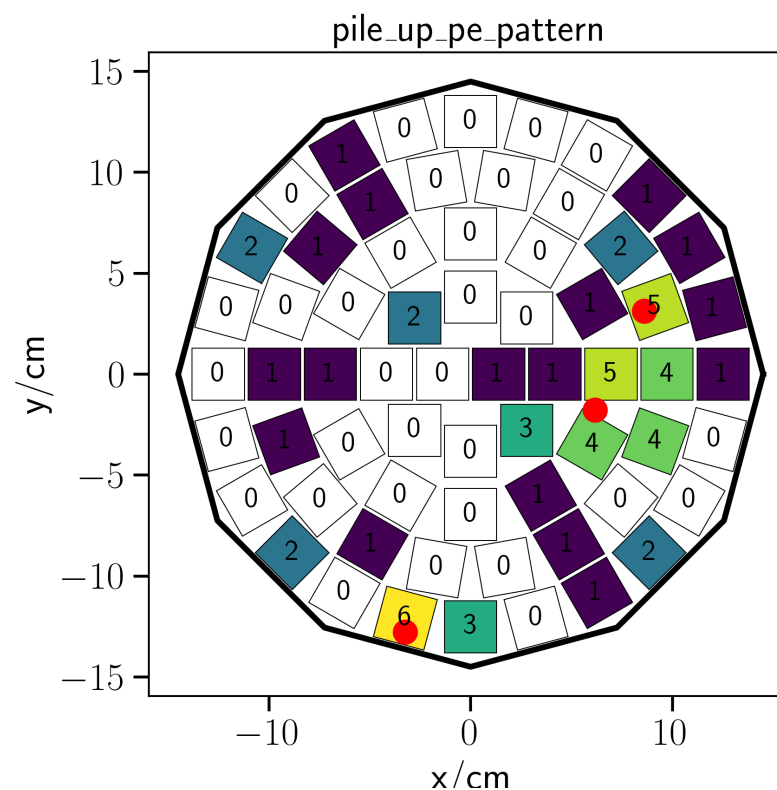
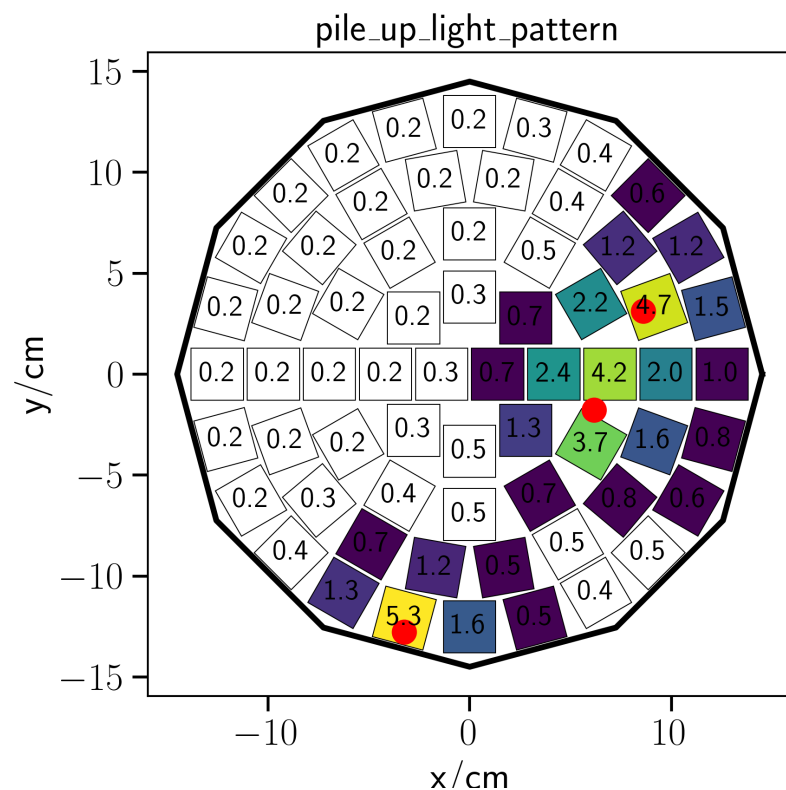
- 得到每个event中pe的随机响应之后，可以计算这个event的`area_pattern`:

```
area_pattern = np.bincount(  
    channel_indices, weights=pe_area, minlength=128)
```

- 最终，我们根据`area_pattern`以及`pe_gain`，折算得到`pile_up_pe_by_area`并只保留其中面积在选定的模拟区间内的事件（90-240pe），并且只保留`pile_up_result`中面积符合要求的事例结果



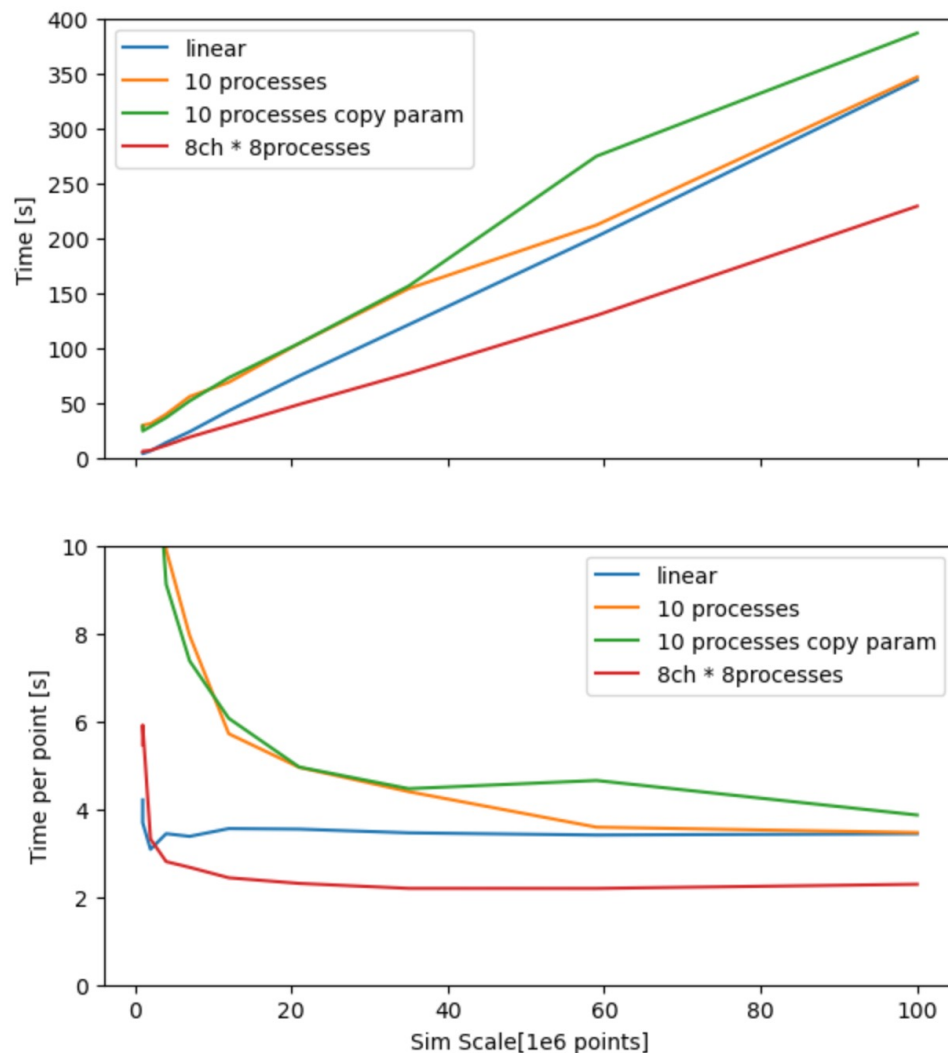
GENERATE_DE_PATTERN()



不同的S2 pattern, 认为Single PE Gain为 $6e6$, Sigma为 $2.4e6$



GENERATE_DE_PATTERN()



- S2 pattern的模拟是之前模拟中运算资源主要消耗的环节，需要对大量的坐标进行插值；
- 在没有启用gpu的情况下，采用 multiprocessing模块希望多进程加快模拟速度，但是python中对于多个进程之间传递大参数（电子坐标）非常缓慢，几乎达到每个进程本身的运行时间，考虑到整体模拟时的效率，最后选择在单批次内部不针对多个通道的模拟过程进行多进程；

Fig6. S2 pattern 不同模拟方案的速度



DE_POSITION_RECON()

- 输入: `pe_by_area[:, :64]` 顶部PMT阵列各个通道实际面积
- 输出: `pile_up_pos_recon`
- 内容: 由郭潇然训练的位置重建CNN对于Pile-up DE事件进行位置重建, 获取当前
`pe_by_area_pattern`下的重建位置;



GENERATE_DE_RECON_PATTERN ()

- 输入: `pile_up_pos_recon`
- 输出: `pile_up_recons_light_pattern`
- 内容: 根据`Pile_up DE`事件的重建位置, 模拟该位置下等面积理想的点状事件的S2 pattern;
 - 与`generate_DE_pattern`类似, 通过对128个通道依次进行插值获得所有事件的`pile_up_recons_pattern`, 之后根据每个`Pile_up DE`的事件面积进行放大, 得到`pile_up_recons_light_pattern`



GENERATE_DE_RECON_PATTERN()

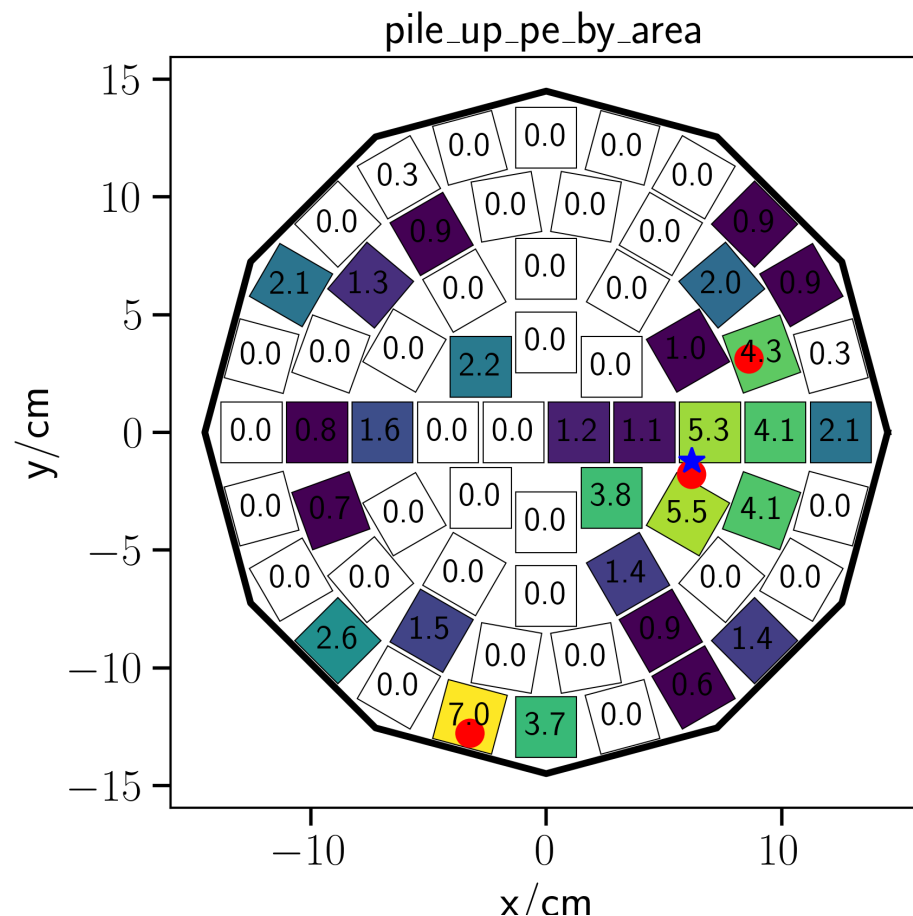


Fig7. Pile_up DE事件模拟实际面积pattern

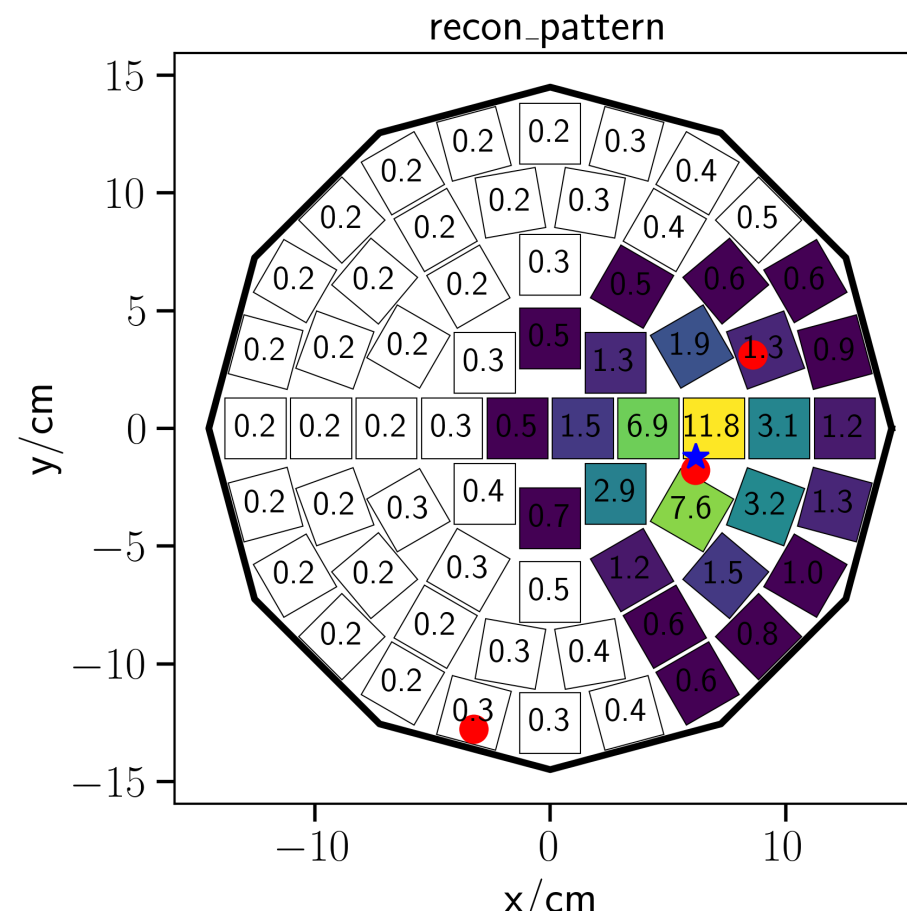


Fig8. Pile_up DE事件重建pattern



DE_SP_COR () 与 DE_PATTERN ()

- 输入: pile_up_result, event_time (muon时间点)
- 输出: pile_up_sp_cor, pile_up_pattern
- 内容: 根据Pile_up DE事件的时间点, 向前找到之前的20个muon event, 计算与其中每个能量沉积点的时空关联概率, 并对所有沉积点求和;

```
fake_prob = (xy_prob * time_prob * point_de_num )
```

```
space_time_cor = np.sum(fake_prob, axis=1)
```

$$P_i = N_i \times (t - t_i)^{-\gamma} \times \frac{1}{2\pi \cdot \sigma^2} e^{-\frac{(x-x_i)^2 + (y-y_i)^2}{2\sigma^2}}$$

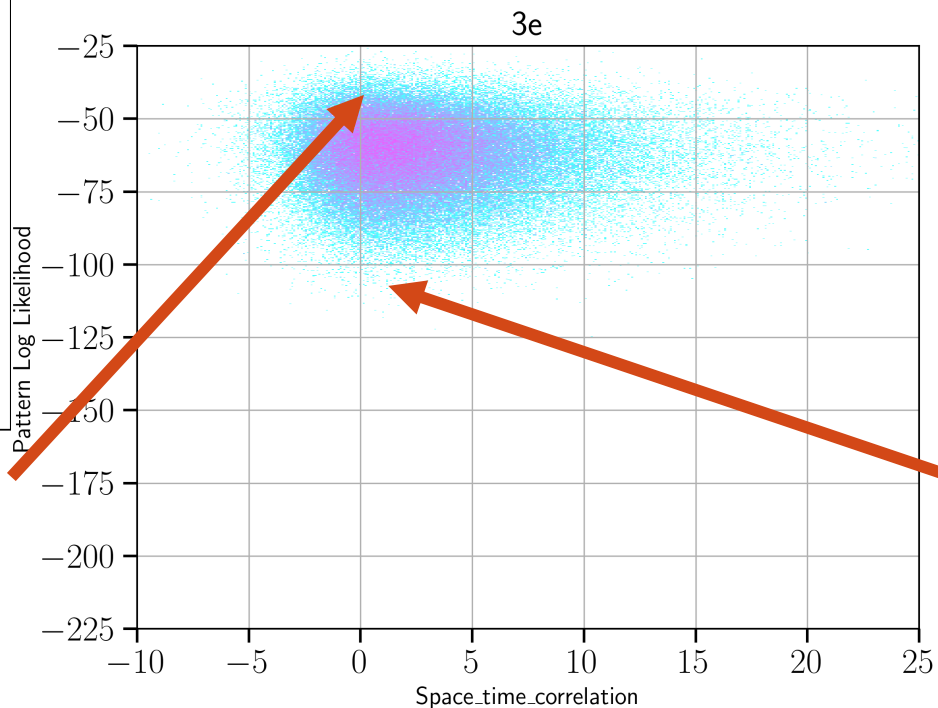
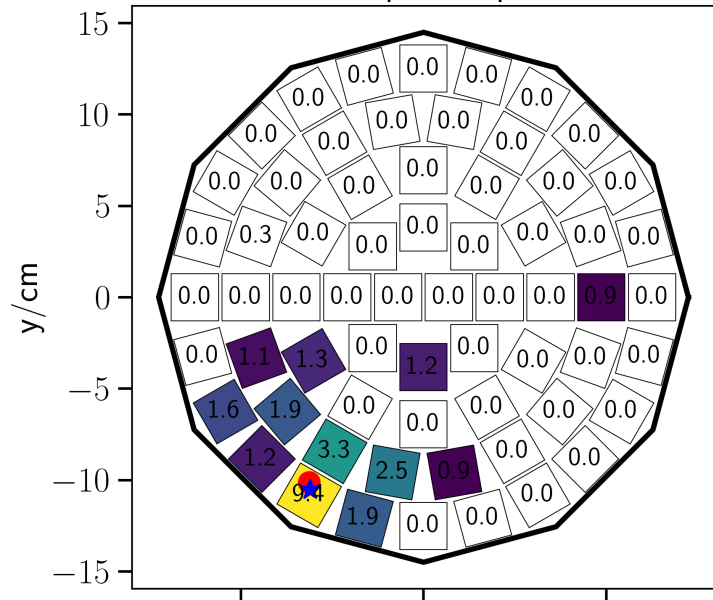
- 根据pile_up_recons_light_pattern和pe_by_area顶部通道的比较, 计算Pattern Likelihood Coefficient

$$P = \sum_{ch=0}^{63} \log \left(\frac{\lambda_{ch}^{N_{ch}^{pe}} \times e^{-\lambda_{ch}}}{N_{ch}^{pe}!} \right)$$

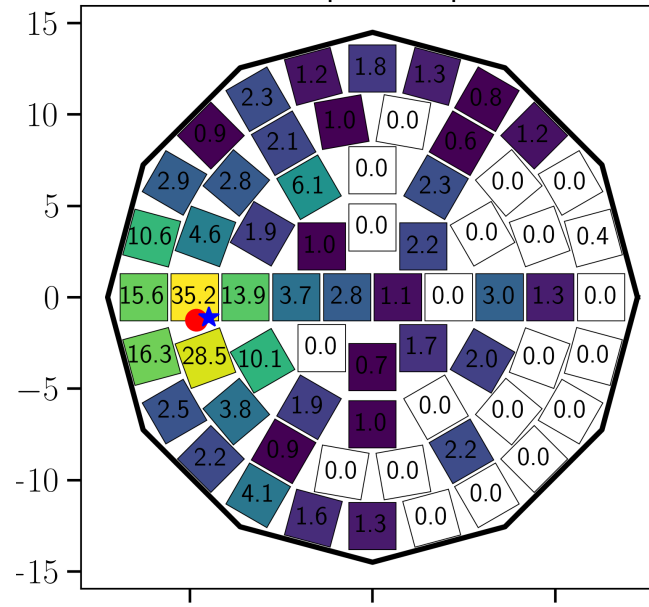
CEVNS模拟

- 与Pile-up DE事件模拟流程类似, 包括模拟pattern, 重建位置, 再次模拟, pattern比较:
 - `generate_CEVNS_points()`
 - `generate_CEVNS_pattern()`
 - `CEVNS_position_recon()`
 - `generate_CEVNS_recon_pattern()`
 - `CEVNS_sp_cor()`

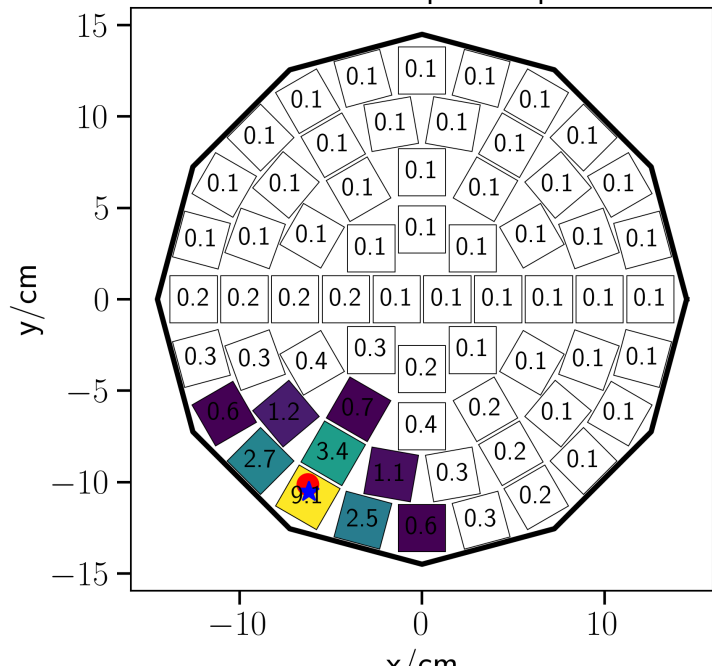
CEvNS_pattern_point



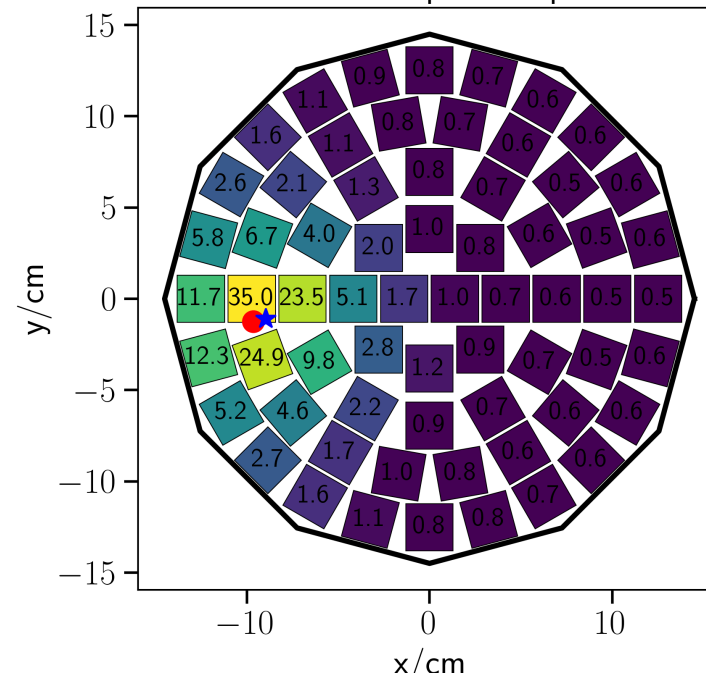
CEvNS_pattern_spread



CEvNS_recon_pattern_point



CEvNS_recon_pattern_spread



左侧 Pattern Coefficient 为-29.11,
三电子事件
右侧为-102,
九电子事件

PATTERN CLASSIFIER

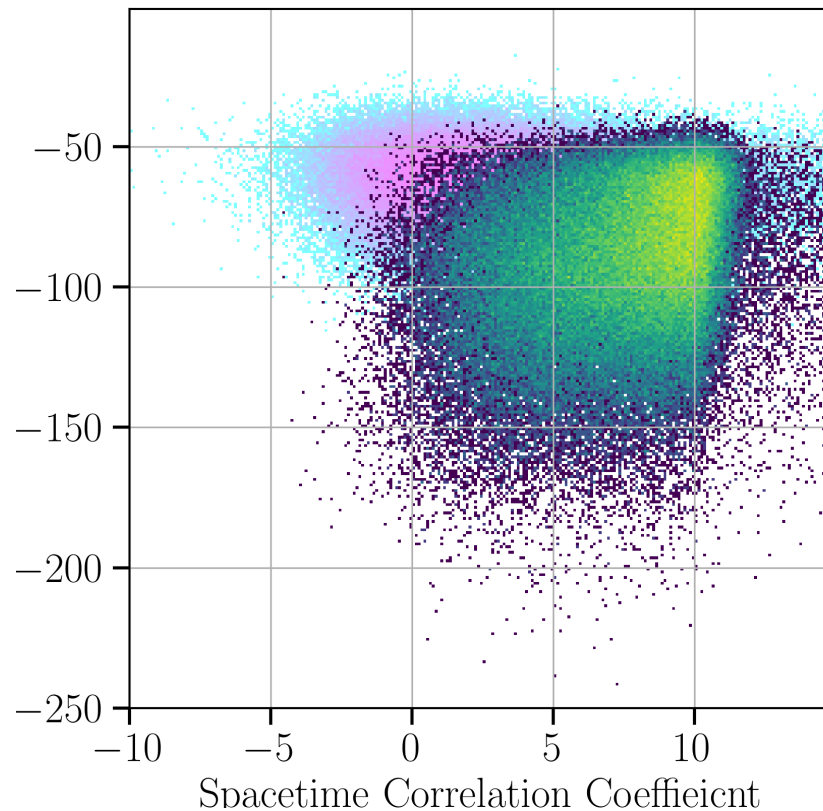


Fig9. Pattern Coefficient与Space Time Correlation Coefficient分布示意图, S2 area>90pe

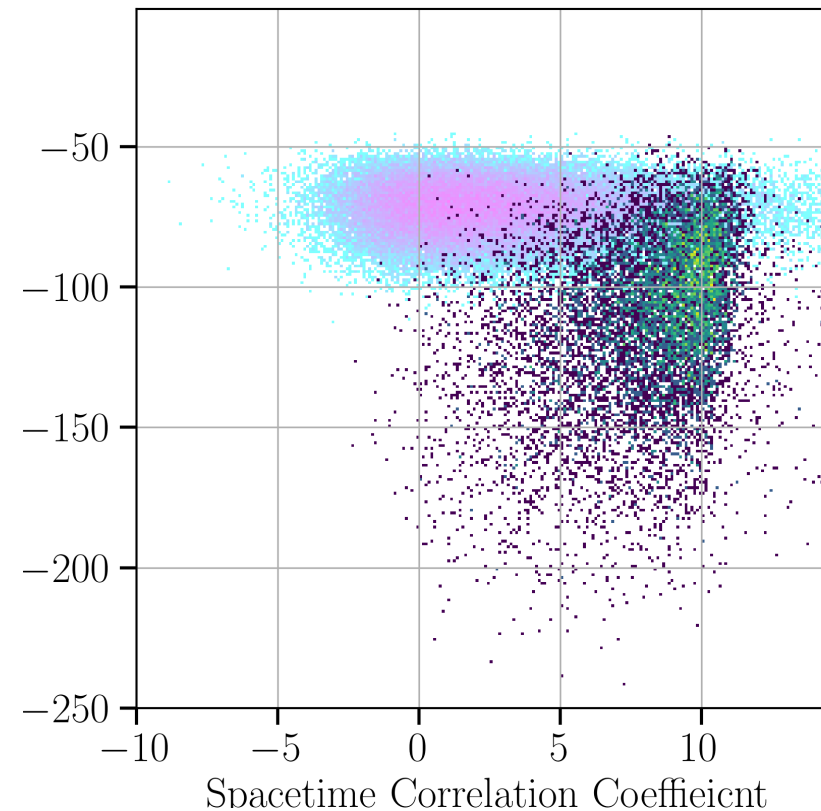


Fig9. Pattern Coefficient与Space Time Correlation Coefficient分布示意图, S2 area>120pe



PATTERN CLASSIFIER

$$Score = P - (k_{st} \cdot C_{space-time}) + k_{area} \cdot S2_{area} + b$$

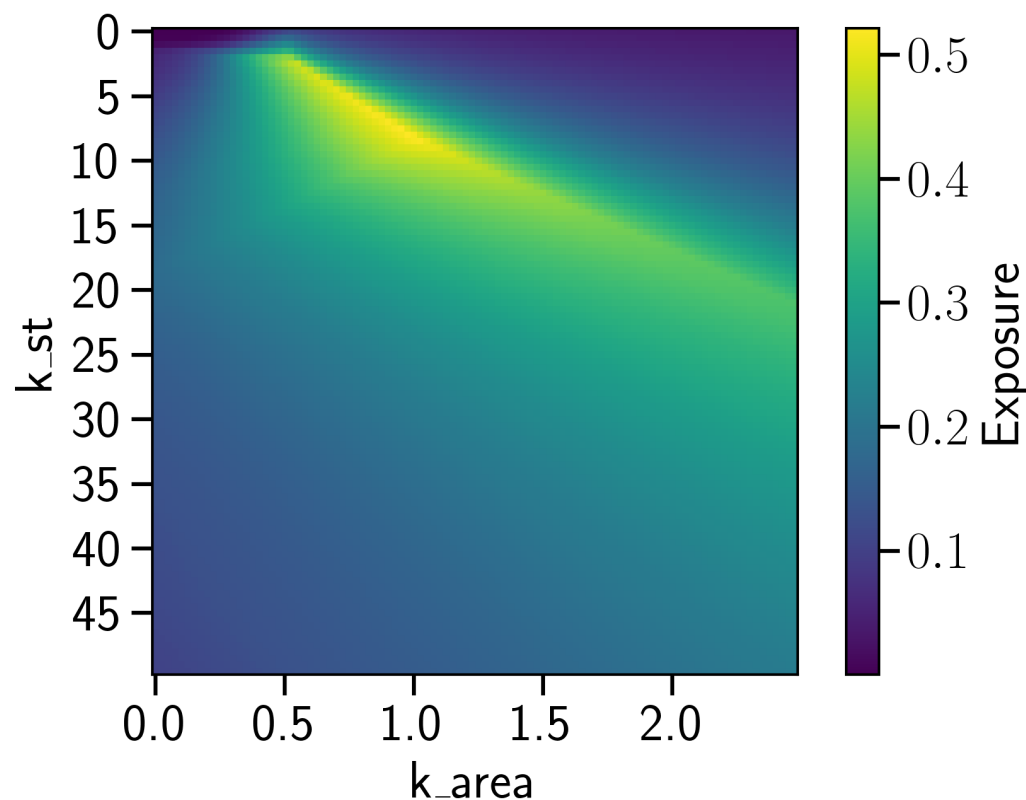


Fig11. 对于S2 area 在[120,240]pe区间内的cevns事件, 要求每个参数组合下延迟电子rejection power 至少为1e5, CEvNS事件保留效率

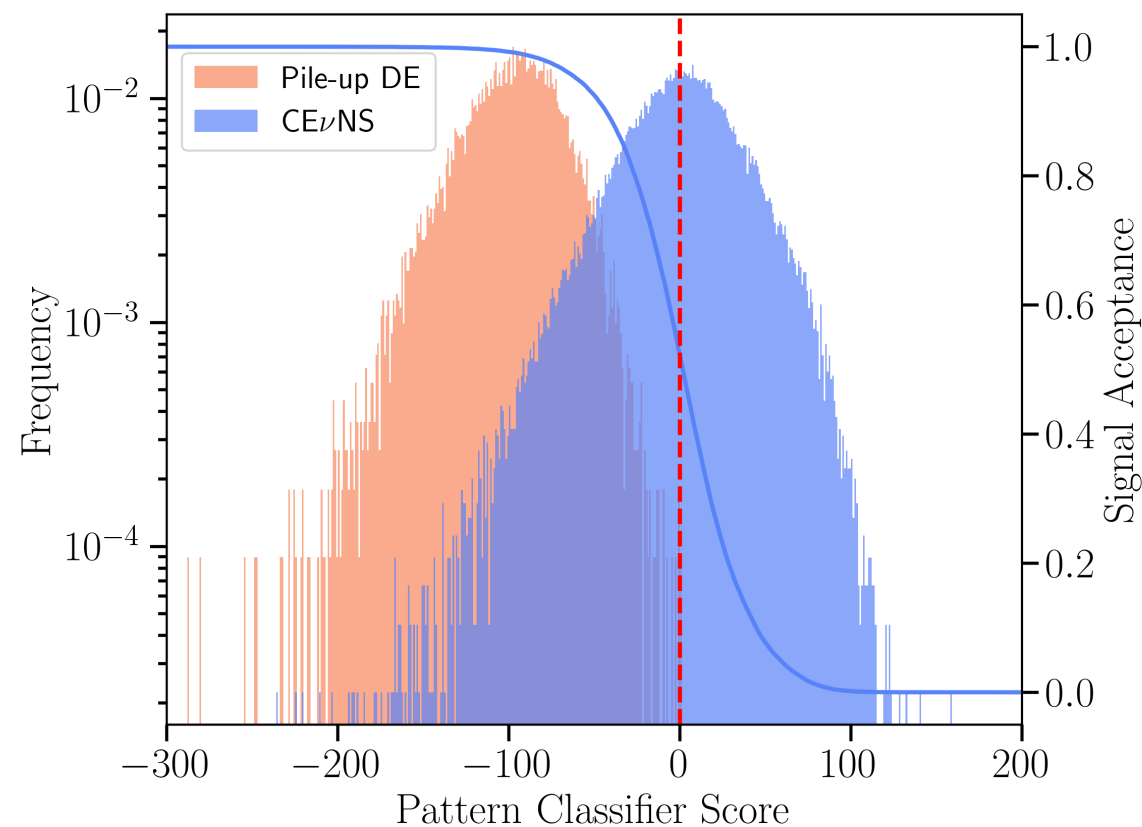


Fig12. Pile_up DE事件与CEvNS事件的分数分布